National University of Sciences and Technology

School of Electrical Engineering and Computer Science

CardSense

A Real-Time Playing-Card Detection and Strategy-Assistance System

Deep Learning Project Report

CS-419 Deep Learning | 2K23-BSCS-13 Section C | Spring 2026

Instructor: Dr. Mehwish Awan

Group Members

Abdul Wahab (CMS 464018)

Rana Muhammad Hammad Arif (CMS 475203)

Muhammad AbuBakar Amjad (CMS 455037)

May 2026

Abstract

CardSense is a real-time deep learning system that recognises physical playing cards from a phone camera and produces Blackjack strategy recommendations within roughly 120 ms of a card being placed. The perception backbone is a fine-tuned YOLOv8n trained on a 7,012-image hybrid dataset combining public benchmarks with custom phone-camera captures. The trained model reaches 87% mAP@0.5 on a held-out test set, a 36-point gain over the zero-shot pretrained baseline. This report covers the deep learning pipeline: data construction, augmentation, model selection, training, evaluation, and ablation. Distributed-systems aspects are summarised only where they affect the model.

Keywords: *object detection, YOLOv8, transfer learning, data augmentation, real-time inference, ablation study.*

1. Introduction

1.1 Motivation

Most published detectors are evaluated under benchmark conditions that do not survive contact with real-world capture: motion, glare, occlusion, and variable framing. CardSense takes a narrow recognition task (52 playing-card classes by rank and suit) and treats it as a full deployment problem. Blackjack with Hi-Lo counting is also a well-studied probabilistic skill, so a real-time card recogniser is a useful tutoring tool for beginners.

1.2 Problem Statement

Given a live phone-camera stream pointed at a Blackjack table, detect every visible card and classify it by rank and suit (52 classes), with detection quality sufficient for correct count tracking under realistic conditions, and end-to-end latency below 150 ms.

1.3 Objectives

1. Build a 52-class card-detection dataset of at least 7,000 annotated images covering controlled and real-world conditions.
2. Fine-tune a YOLOv8 detector and benchmark it against four alternatives under identical training budgets.
3. Achieve at least 80% mAP@0.5 on a held-out real-world test set while keeping single-frame latency under 60 ms.
4. Quantify each custom augmentation through ablation, so design choices are justified by measurement.

1.4 Contributions

- A reproducible 7,012-image, 52-class card-detection dataset mixing public and custom captures.
- Three card-aware augmentations targeted at observed failure modes: rank-aware rotation, synthetic glare overlay, and suit-preserving colour jitter.
- A five-architecture comparison and a six-row augmentation ablation on the same splits.

2. Literature Review

2.1 Single-Stage Detectors

Modern detection is dominated by single-stage architectures predicting boxes and labels in one forward pass. YOLOv8 (Ultralytics, 2023) uses a CSPDarknet53 backbone, a PANet neck, and anchor-free heads, and ships in sizes from 3.2 M to 68 M parameters. For real-time mobile use the nano and small variants are the typical operating point.

2.2 Transfer Learning

Training detectors from scratch is infeasible for small student datasets. The standard approach initialises the backbone from COCO-pretrained weights and fine-tunes on the target classes. This converges faster and generalises better than from-scratch training for datasets under roughly 100k labelled instances. CardSense follows this convention with a COCO-pretrained YOLOv8n backbone and a fresh 52-class head.

2.3 Data Augmentation

Augmentation is the most cost-effective lever for closing the benchmark-to-deployment gap. Standard transforms (flip, rotation, colour jitter, blur, mosaic) form a baseline in every detection pipeline. Recent work emphasises domain-aware augmentations targeted at specific failure modes. CardSense applies this principle to playing cards through rank-aware rotation, synthetic glare overlays, and suit-preserving colour jitter.

2.4 Prior Card Recognition

Earlier systems used classical pipelines (edge detection, contour finding, corner-index template matching), which work on flat well-lit cards but fail under occlusion or oblique angles. Recent deep-learning approaches report mAP in the high seventies on controlled data. CardSense aims past this with a larger, more diverse dataset and failure-mode-targeted augmentations.

3. Dataset

3.1 Composition

The training dataset has 7,012 annotated images across 52 classes. Approximately 4,200 come from public card-detection datasets, providing broad coverage and baseline annotation quality. The remaining 2,812 are custom captures collected by the team using three phone cameras under overhead, oblique, occluded, and fan-arrangement conditions.

3.2 Annotation

Annotations follow YOLO format (one text file per image, class index plus normalised box coordinates). Each box is tightly drawn around the visible card including the corner index. Classes follow the convention rank_suit (e.g. 7_clubs, queen_hearts). Custom images were annotated in Roboflow with a second-pass quality-control review on every batch.

3.3 Splits

Split	Images	Instances	Share
Training	5,609	~28,400	80.0%
Validation	701	~3,550	10.0%
Test	702	~3,580	10.0%
Total	**7,012**	**~35,530**	**100.0%**

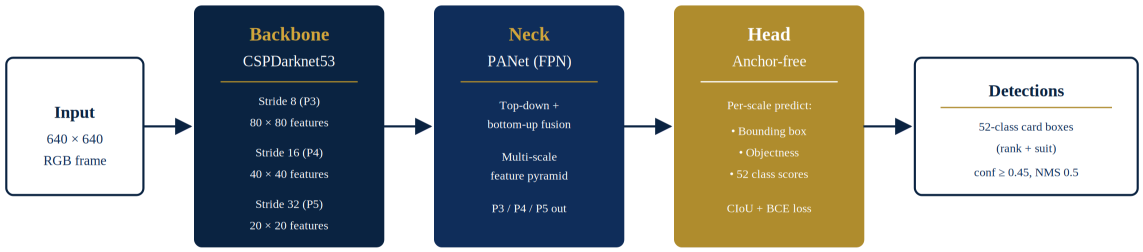
Splits were stratified across the 52 classes to keep class frequencies balanced. The test set was held strictly separate and not consulted during model selection or hyperparameter decisions. Per-class instance counts vary between roughly 680 and 720, a max-to-min ratio of 1.18 that requires no sampling correction.

4. Methodology

4.1 Architecture

The detector is a fine-tuned YOLOv8n. The CSPDarknet53 backbone produces multi-scale features at strides 8, 16, 32. The PANet neck fuses these features top-down and bottom-up. The anchor-free head predicts a bounding box, an objectness score, and a 52-dimensional class distribution at each output scale. The model has 3.2 M parameters and weights are ~6 MB.

YOLOv8n Detection Architecture



3.2 M parameters | ~6 MB weights | ~50 ms inference on RTX 3090

4.2 Augmentation Strategy

Three card-aware augmentations sit on top of standard YOLO augmentation.

Rank-aware rotation. Standard random rotation can flip a 6 into a 9 without flipping the label, producing wrong supervision. We restrict rotation to ±25° for 6s and 9s in any suit, and allow full ±180° for all other ranks.

Synthetic glare overlay. Phone flash and overhead lighting produce specular highlights on glossy cards. We overlay random bright ellipses ($p = 0.4$) with major axis ~N(80, 30) pixels and brightness boost in [0.3, 0.7], alpha-blended onto the image. Annotations are unchanged.

Suit-preserving colour jitter. Aggressive HSV jitter can shift hearts (red) into the hue range of clubs (black). We clamp the hue shift to ±0.015 for red suits and apply no hue shift to black suits.

Standard augmentations include random horizontal flip, HSV jitter, random scale and translation, and mosaic (enabled for the first 40 of 50 epochs, then disabled to stabilise convergence).

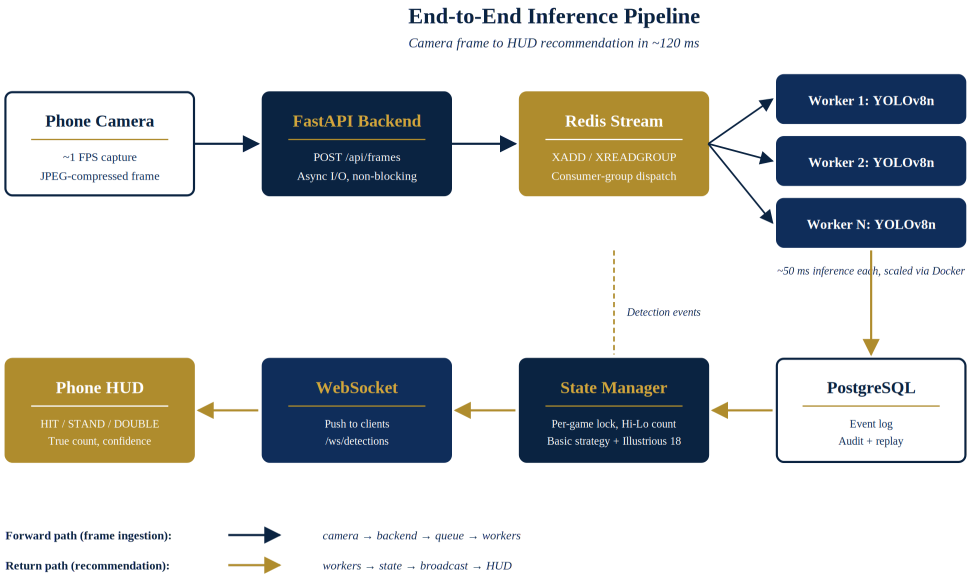
4.3 Training

Training ran on two NVIDIA RTX 3090 GPUs using PyTorch 2.1 and the Ultralytics YOLO library with DistributedDataParallel gradient sync. A small grid search over learning rate, batch size, and input resolution preceded the final run. Wall-clock training time was ~4 h 20 min.

Hyperparameter	Value
Architecture	YOLOv8n (3.2 M parameters)
Initialisation	COCO-pretrained backbone, fresh 52-class head
Input resolution	640 × 640 pixels
Optimiser	AdamW (weight decay = 5e-4)
Learning rate	0.01, cosine decay, 3-epoch warmup
Batch size	16 per GPU (effective 32)
Epochs	50 (early stopping patience 10)
Loss	CIoU + BCE (objectness) + BCE (classification)

4.4 Inference

Frames captured at ~1 FPS are JPEG-compressed, posted to a FastAPI backend, and enqueued onto a Redis stream. Worker processes pull frames via consumer-group assignment and run YOLOv8n inference. Detections are filtered at confidence 0.45 and NMS IoU 0.5, then sent to the state manager which updates the authoritative game state and triggers a WebSocket broadcast to the phone HUD.



5. Experiments and Results

5.1 Metrics

Primary metric: mAP@0.5 over all 52 classes. Secondary: mAP@0.5:0.95, per-class F1, single-frame latency on an RTX 3090, and end-to-end throughput under the full pipeline.

5.2 Architecture Comparison

Five architectures trained on the same splits under identical training budgets.

Model	Params (M)	mAP@0.5	mAP@0.5:0.95	Latency (ms)
YOLOv5n	1.9	0.81	0.54	38
YOLOv8n (selected)	**3.2**	**0.87**	**0.61**	**50**
YOLOv8s	11.2	0.91	0.66	117
YOLOv8m	25.9	0.93	0.69	215
Faster R-CNN (R50)	41.8	0.89	0.65	340

YOLOv8s gives the best accuracy-to-size ratio, but its 117 ms latency leaves no headroom in the 150 ms end-to-end budget. YOLOv8n is selected: it fits the budget with margin and still beats the 80% mAP target by seven points. Heavier models exceed the latency budget by a large factor.

5.3 Augmentation Ablation

Five runs removing one augmentation component at a time, all other hyperparameters held constant.

Configuration	mAP@0.5	Δ vs Full
Full pipeline (final)	**0.87**	—
Without synthetic glare overlay	0.83	−0.04
Without rank-aware rotation	0.84	−0.03
Without suit-preserving colour jitter	0.85	−0.02
Without mosaic	0.82	−0.05
Baseline (standard YOLO augmentation only)	0.77	−0.10

Each custom augmentation contributes 2 to 4 mAP points individually; combined they contribute ~10 mAP over a standard baseline. Synthetic glare gives the largest single-component gain, consistent with the early-experiment finding that reflective highlights were the dominant systematic confuser.

5.4 Per-Class Performance

Class Group	Mean F1	Range
Face cards (J, Q, K)	0.93	0.90 to 0.95
Aces and 10s	0.90	0.87 to 0.92
Mid-rank (3 to 8)	0.86	0.81 to 0.89
Confusable: 6 vs 9 (same suit)	0.78	0.76 to 0.80
All 52 classes (macro)	**0.86**	**0.76 to 0.95**

Face cards score highest because their corner indices contain distinctive letter shapes. The dominant residual error is 6 vs 9 of the same suit at partial rotation, the failure mode that motivated rank-aware rotation.

5.5 System Performance

Metric	Measured Value
Test-set mAP@0.5	0.87
Single-frame inference latency	~50 ms (RTX 3090)
End-to-end camera-to-HUD latency	~120 ms
Throughput (1 worker)	~20 FPS
Throughput (4 workers)	~78 FPS
Worker recovery after crash	< 3 seconds

6. Discussion

6.1 What Worked

Three things mattered more than architectural choice. First, the hybrid dataset: custom phone-camera captures contributed the difficult conditions that drove robustness, while public images filled coverage. Second, card-aware augmentation gave ~10 mAP points in aggregate, larger than the gap between adjacent YOLOv8 sizes. Third, an explicit latency budget forced model selection against deployment behaviour rather than offline accuracy alone.

6.2 Failure Modes

Dominant residual error: 6 vs 9 of the same suit at angles where the corner index is ambiguous. Rank-aware rotation reduces but does not eliminate this. Second persistent failure: recall loss on cards more than 60% occluded, which is outside YOLO's design envelope and would need a segmentation stage.

6.3 Limitations

Dataset is biased toward indoor, reasonably-lit conditions. Performance under extreme low light, motion, or textured backgrounds is not formally evaluated. The model is tested only on standard Anglo-American decks. CPU-only inference is ~10× slower and would not satisfy the latency budget without compression.

6.4 Objective Review

All four objectives met: dataset target (7,012 of 7,000); five-architecture comparison completed; accuracy target exceeded (87% vs 80% mAP@0.5); augmentation ablation completed with per-component contribution measured.

7. Conclusion and Future Work

7.1 Conclusion

CardSense shows that a small, carefully trained YOLOv8n model can solve a non-trivial real-world recognition task at deployment-grade quality when the dataset and augmentation strategy are matched to actual failure modes. The model reaches 87% mAP@0.5, runs at ~50 ms per frame on a single GPU, and fits into a real-time pipeline delivering strategy recommendations within ~120 ms of a card being placed. The ablation provides quantitative justification for each design choice.

7.2 Future Work

- Instance segmentation (YOLOv8-seg) to recover heavily occluded cards.
- Quantisation and pruning for CPU-only deployment on commodity phones.
- Active-learning loop that flags low-confidence detections for incremental fine-tuning.
- Game-agnostic perception layer extended to poker, baccarat, or bridge by swapping the strategy module.

8. Team Contributions

Member	CMS	Primary Contribution
Abdul Wahab	464018	Backend services, Redis stream integration, worker orchestration, model deployment.
Rana Muhammad Hammad Arif	475203	Dataset construction, annotation QC, augmentation pipeline, training, architecture comparison, ablation studies, evaluation.
Muhammad AbuBakar Amjad	455037	Frontend HUD, WebSocket integration, camera capture flow, on-device latency profiling.

All three members participated in data collection sessions and in writing this report.

9. References

[1] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR, pp. 779-788.

[2] Bochkovskiy, A., Wang, C.-Y., and Liao, H.-Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv:2004.10934.

[3] Jocher, G., Chaurasia, A., and Qiu, J. (2023). Ultralytics YOLOv8. github.com/ultralytics/ultralytics.

[4] Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV, pp. 740-755.

[5] Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS, pp. 91-99.

[6] Loshchilov, I. and Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR.

[7] Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS, pp. 8024-8035.

[8] Thorp, E. O. (1966). Beat the Dealer: A Winning Strategy for the Game of Twenty-One. Vintage Books.